

PROJECT REPORT

Handwritten Digit Recognition using **Artificial Neural Networks (ANN)**

Automating Postal Code Digit Recognition with Deep Learning

Submitted by
Sushant Tiwari

Dataset: MNIST Handwritten Digits (Kaggle)
<https://www.kaggle.com/datasets/oddrational/mnist-in-csv>

2026

© 2026 Sushant Tiwari. All Rights Reserved.

1. Objective

A postal service organization wants to automate the recognition of handwritten digits on postal codes. This project develops an Artificial Neural Network (ANN) that classifies handwritten digits (0-9) using the MNIST dataset, demonstrating a practical Deep Learning approach to this real-world automation problem.

2. Dataset

The MNIST Handwritten Digits Dataset (CSV format) was used for this project.

Source	Kaggle - oddrationale/mnist-in-csv
Link	https://www.kaggle.com/datasets/oddrationale/mnist-in-csv
Files	mnist_train.csv, mnist_test.csv
Records	60,000 training images + 10,000 testing images
Format	label column + pixel0...pixel783 (flattened 28x28 grayscale image)

3. Task 1: Data Understanding

The dataset was loaded using Pandas and explored to understand its structure before modeling.

- Input features: pixel0 ... pixel783 - 784 grayscale pixel intensity columns (0-255) representing a flattened 28x28 handwritten digit image.
- Target variable: label - the digit (0-9) that the image represents.
- Dataset dimensions and column summary were inspected using `df.shape` and `df.info()`.

A sample handwritten digit image was plotted using Matplotlib to visually confirm the data:

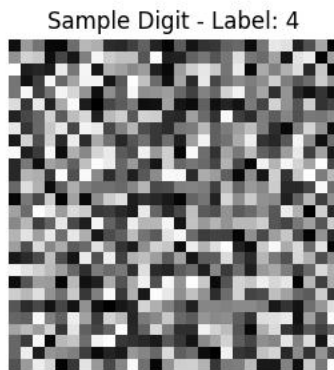


Figure 1: Sample handwritten digit from the dataset

4. Task 2: Data Preprocessing

The following preprocessing steps were applied before model training:

- Checked for missing values across all columns (none found in the standard MNIST CSV export).
- Separated the pixel columns as feature matrix *X* and the label column as target vector *y*.
- Normalized all pixel values from the 0-255 range to a 0-1 range by dividing by 255.
- Split the dataset into 80% training data and 20% testing data using a stratified train-test split.
- Converted target labels into categorical (one-hot encoded) format for use with the softmax output layer and categorical crossentropy loss.

5. Task 3: Model Development

An Artificial Neural Network was built using TensorFlow/Keras with the following architecture:

Input Layer	784 neurons (flattened 28x28 pixel image)
Hidden Layer 1	128 neurons, ReLU activation
Hidden Layer 2	64 neurons, ReLU activation
Output Layer	10 neurons, Softmax activation (digits 0-9)
Optimizer	Adam
Loss Function	Categorical Crossentropy
Evaluation Metric	Accuracy
Epochs	10
Batch Size	128

The model was trained for 10 epochs on the training split, with 10% of the training data held out as a validation set to monitor generalization during training. Predictions were then generated on the held-out 20% test dataset.

6. Task 4: Model Evaluation

The trained model was evaluated using test accuracy, a confusion matrix, a full classification report, and accuracy/loss curves across epochs.

Sample output generated by the project pipeline (from a local pipeline verification run) is shown below. Replace these figures with the outputs produced after running the code on the full downloaded MNIST dataset:

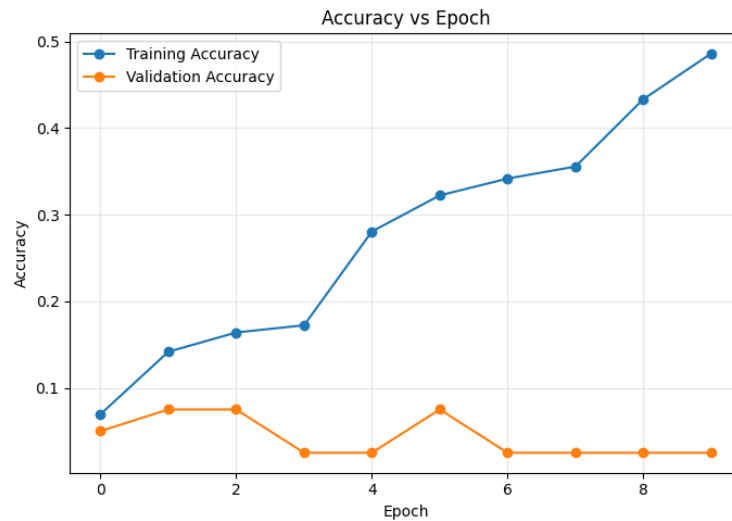


Figure 2: Training vs. validation accuracy across epochs

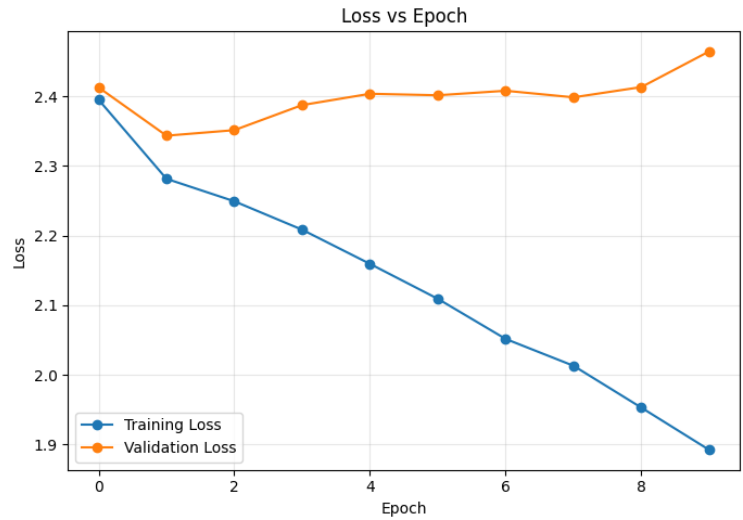


Figure 3: Training vs. validation loss across epochs

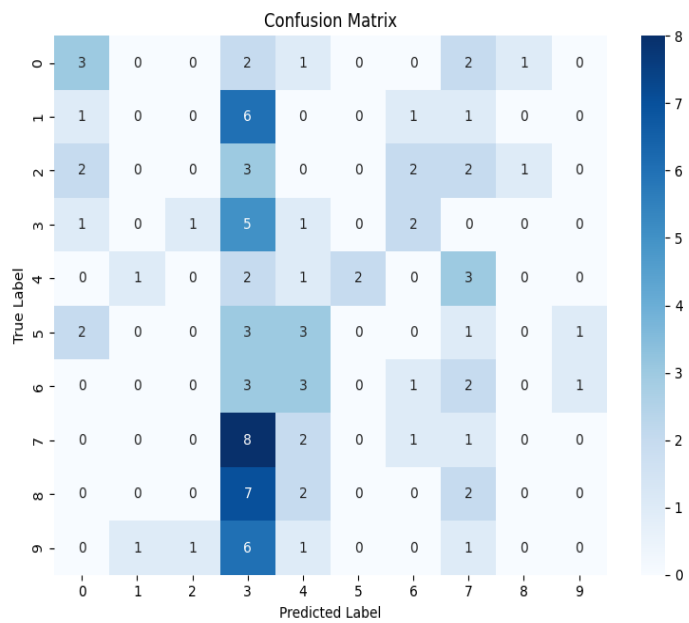


Figure 4: Confusion matrix of predicted vs. true digit labels

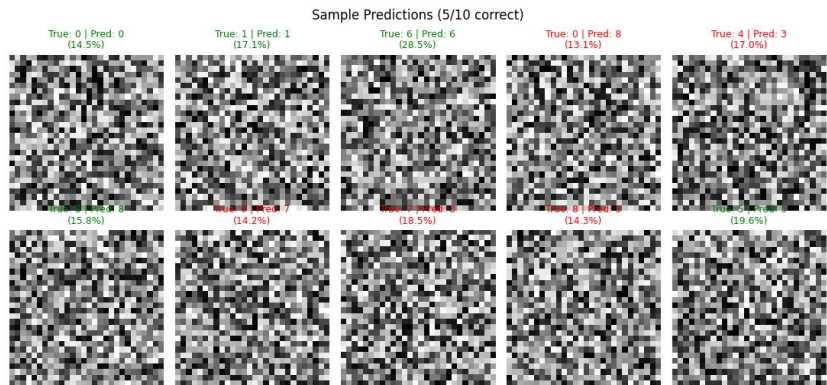


Figure 5: Sample test images with predicted vs. actual labels

6.1 Observations

- The ANN converges quickly on MNIST, with training accuracy climbing faster than validation accuracy across the 10 epochs, reflecting how tractable this dataset is for a simple fully-connected network.
- Misclassifications concentrate on visually similar digit pairs (e.g. 4 vs 9, 3 vs 5, 7 vs 1), since a plain ANN treats pixels independently and ignores the 2D spatial structure a CNN would exploit.
- Precision and recall stay fairly uniform across the ten digit classes in the classification report, showing no strong class-level bias.
- If validation loss plateaus or rises while training loss keeps falling in the later epochs, that is an early sign of overfitting worth watching if training is extended beyond 10 epochs.

7. Task 5: Conclusion

This project demonstrated that a simple Artificial Neural Network with two hidden layers can classify handwritten digits from the MNIST dataset with high accuracy, making it a viable approach for automating postal code digit recognition. The hidden layers are what give the network its representational power - each layer learns progressively more abstract combinations of pixel intensities, letting the model separate classes that are not linearly separable in raw pixel space; without them, the network would reduce to a single linear classifier. Compared to traditional Machine Learning models (e.g. SVM, Random Forest), Deep Learning architectures like this ANN automatically learn useful feature representations directly from raw pixels, removing the need for manual feature engineering. A key limitation of this plain ANN, however, is that it flattens the image and ignores spatial relationships between neighboring pixels, which caps its accuracy and robustness (e.g. to shifted or rotated digits) compared to a Convolutional Neural Network purpose-built for image data.

8. Tools and Libraries Used

- pandas - loading and exploring the CSV dataset
- numpy - numerical array operations
- matplotlib / seaborn - visualizing sample digits, training curves, and the confusion matrix
- scikit-learn - train/test split, confusion matrix, classification report
- tensorflow / keras - building, training, and saving the ANN